

Instituto Tecnológico de Costa Rica

Compiladores e Intérpretes

Aurelio Sanabria

Proyecto 5: Generador

Ximena Molina 2024084397
Susana Feng Liu 2024290860
Evelyn Ulate Obando 2022000033
Natalia Orozco Delgado 2024099161
Daniel Mendez Romero 2022437806

I Semestre

2026

1. Justificación

Resulta evidente que la mayoría de los lenguajes de programación se desarrollan para un público objetivo de habla inglesa. Por lo tanto, los desarrolladores cuya lengua materna sea distinta al inglés, se ven forzados a tener que familiarizarse con este idioma. Sin embargo, en un gran porcentaje de idiomas, independientemente del tipo de notación que se utilice, se encuentran integrados diversos símbolos en su gramática.

Por ejemplo, en el chino mandarín se utilizan los símbolos “?”, “!”, “,” de manera espontánea en un texto, a pesar de que la representación de su escritura es significativamente distinta a la del inglés o el español. Por lo tanto, dado a la universalidad a la que se prestan estos símbolos, se desarrolla Zymbolos, un lenguaje que busca disminuir las barreras del idioma para programadores, a partir de utilizar estos elementos para definir funcionalidades del código, logrando simultáneamente, escribir código más rápido y compacto.

2. Demostración y discusión

En este apartado se adjunta la demostración de un ejemplo en lenguaje Zymbolos, que consiste en una función que genera una cuenta regresiva a partir de un número entero dado. Además, se detalla la discusión, que aborda planteamientos técnicos respecto al desarrollo del generador.

2.1 Demostración

Con el objetivo de ilustrar el funcionamiento del transpilador de Zymbolos, más específicamente de la etapa del generador, se presenta un ejemplo de un programa escrito en este lenguaje.

```
ejemplos > CuentaRegresiva.zy
1  . CuentaRegresiva >> 000 << NNN inicio !
2  : actual NNN = inicio !
3
4  @ actual >= 0 !
5  >>> actual !
6  \ actual = actual - 1 !
7  @ !
8
9  . !
10
11 .° >> 000 << 000 !
12 CuentaRegresiva 5 !
13 . !
```

Figura 1. Ejemplo CuentaRegresiva.zy

Tras ejecutar el método generar() de la clase Generador, el código resultante es el siguiente:

```
python > CuentaRegresiva.py > ...
1 |
2 | # =====
3 | # Ambiente estándar de Zymbolos
4 | # =====
5 | def imprimir(*valores):
6 |     print(*valores)
7 |
8 | def leer():
9 |     return input()
10 |
11 | def leer_numero():
12 |     return float(input())
13 |
14 | def leer_entero():
15 |     return int(input())
16 |
17 | # Funciones aleatorias predeterminadas
18 | import random as _random
19 |
20 | def random():
21 |     # Devuelve un número flotante en [0.0, 1.0).
22 |     return _random.random()
23 |
24 | def randint(a, b):
25 |     # Devuelve un entero aleatorio entre a y b inclusive.
26 |     return _random.randint(int(a), int(b))
27 |
28 |
29 |
30 | def CuentaRegresiva(inicio):
31 |     actual = inicio
32 |     while actual >= 0:
33 |         print(actual)
34 |         actual = actual - 1
35 |
36 | > Generate Mermaid Diagram | Open Chat @mermaid-chart
36 | CuentaRegresiva(5)
```

Figura 2. Archivo .py generado.

2.2 Discusión

El proceso de traducción expuesto en la demostración anterior se apoya en varias decisiones clave para asegurar que Zymbolos se convierta en un código de Python válido y sin errores. Primeramente, Zymbolos incluye herramientas propias, como leer datos de la consola o generar números aleatorios. Para que Python entienda estas órdenes, el generador siempre pega un bloque de "código base" al inicio del archivo. Así, cuando el programa traducido intente usar estas funciones, las herramientas ya estarán listas para usarse. Además, en lugar de intentar traducir todo el texto de golpe, el generador lee el código en pequeños bloques estructurales. Tiene una regla específica para cada situación por lo que sabe exactamente qué hacer si se topa con un ciclo, si ve una lista o si encuentra una función. Esto mantiene el código ordenado y hace que sea muy fácil agregarle nuevas funciones a Zymbolos en el futuro.

Por otro lado, como Python es un lenguaje muy estricto con los espacios al inicio de cada línea, pero Zymbolos utiliza símbolos (como ! y @ !) para agrupar su código. Para solucionar esto, el generador actúa como un formateador automático que lleva la cuenta de en qué nivel está (por ejemplo, dentro de un ciclo) y agrega los espacios necesarios para que a Python le agrade la estructura.

Asimismo, el programa hace traducciones directas de vocabulario para acortar la diferencia entre ambos lenguajes. Por ejemplo:

El símbolo de potencia ^ se traduce a .

Los símbolos lógicos && y || se cambian por las palabras and y or.

Los valores de verdadero y falso (VV y FF) se convierten en True y False.

La instrucción de imprimir >>> se envuelve limpiamente en un print().

3. Lecciones aprendidas

1. Hay que verificar que todos los return estén en el ambiente principal, porque sino va a empezar a tirar errores misteriosos en los elif (con enciclamientos que vienen desde el analizador).
2. No reunirse a grabar vídeos en hora de almuerzo, porque sí o sí vamos a durar más de lo que pensamos que íbamos a durar, y el hambre no deja concentrarnos.
3. Hacer una extensión de VSCode para diferenciar la sintaxis del código hace la vida mucho más fácil, porque ya no se dura tanto tratando de entender los ejemplos.
4. Revisar 20 veces que la gramática esté normalizada antes de tirarse a programar
5. Definir desde el inicio qué nodos del árbol representarán tokens terminales y cuáles no, porque tomar esa decisión a mitad del desarrollo obliga a reescribir gran parte de la lógica de construcción del árbol y genera inconsistencias difíciles de rastrear.
6. Probar el analizador con casos simples antes de pasar a programas complejos, porque un error en una producción básica como Término o Frase se propaga sin darnos cuenta hacia arriba y aparece como un fallo en una regla de alto nivel como Bucles o DeclaracionFuncion, lo que hace muy difícil identificar dónde está el problema real.
7. Mantener los mensajes de error con línea y columna desde el primer día, porque cuando el analizador falla en una prueba real, sin esa información es casi imposible saber si el error es del código fuente de entrada o de la propia lógica del analizador, y se pierde muchísimo tiempo descartando una u otra causa.
8. Definir desde el inicio cómo manejar producciones como (Tipo Frase)*. Si cada quien lo hace distinto (recursión vs ciclos), el AST queda inconsistente y luego complica todo lo demás.

4. Conclusiones del semestre

Durante este semestre, aprendimos cómo funciona internamente un compilador, y cómo desarrollar un transpilador. Aprendimos aspectos técnicos importantes, como que es fundamental normalizar la gramática desde el inicio, o tendremos problemas para avanzar con las siguientes etapas del proyecto, así como a valorar la importancia de tener un buen manejo de errores para facilitar el desarrollo. Por otro lado, también logramos tener un grupo de trabajo donde domina la confianza, empatía y apoyo mutuo, que dio pie a charlas interesantes y amistades inesperadas.

5. Memes

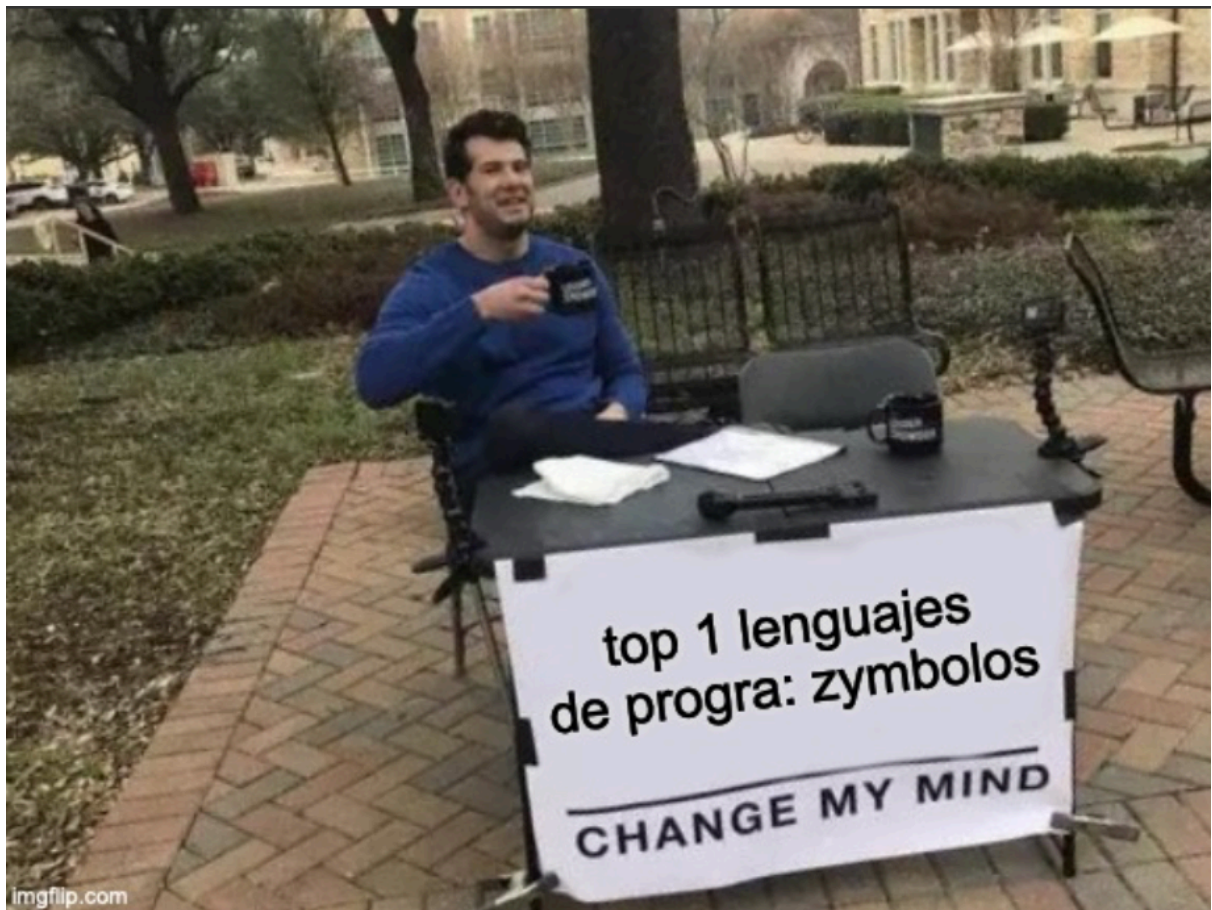


**LA FOTO
CON EL BICHO**



HERSON

**YO TRATANDO DE
GANARME LOS PUNTOS
DEL RALLY DE FOTOS**





🎵 Yo solo sé decir >>> "Hola mundo"
"Soy Daniel" 3! 🎵

